

EXPERIMENT – 12.1

Name: K.anshu

Roll No: 2520030440

Title

Producer-Consumer Problem using POSIX Threads and Semaphores

Aim

To implement the producer-consumer problem using POSIX threads and counting semaphores for synchronization.

Theory

The producer-consumer problem is a synchronization problem in which a producer generates items and places them into a shared buffer, while a consumer removes items from the buffer. Counting semaphores track the number of empty and filled buffer positions. A mutex protects the shared buffer from simultaneous access by multiple threads.

Algorithm

1. Initialize the buffer.
2. Initialize the empty semaphore with the buffer size and the full semaphore to zero.
3. Initialize the mutex.
4. Create producer and consumer threads.
5. The producer waits for an empty position, inserts an item, and signals that a position is filled.
6. The consumer waits for a filled position, removes an item, and signals that a position is empty.
7. Wait for both threads to complete.
8. Destroy the semaphores and mutex.

Program Code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;
```

```

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main()
{
    pthread_t producer_thread, consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Execution and Screenshots

Screenshot 1 – Program Code (Producer)

```

admin@heeraaj@DESKTOP-J39GHP7:~$ cat > exp12_producer_consumer.c <<'EOF'
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
    }
}

```

Screenshot 2 – Consumer Function

```

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread, consumer_thread;
}

```

Screenshot 3 – Main Function and End of Code

```

int main()
{
    pthread_t producer_thread, consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
EOF

```

Screenshot 4 – Compilation and Execution Output

```

admin@heeraaj@DESKTOP-J39GHP7:~$ gcc exp12_producer_consumer.c -o exp12_producer_consumer -pthread
admin@heeraaj@DESKTOP-J39GHP7:~$ ./exp12_producer_consumer
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10

```

Observed Output

```

Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10

```

```
Consumed: 6  
Consumed: 7  
Consumed: 8  
Consumed: 9  
Consumed: 10
```

Result

The producer-consumer problem was implemented using POSIX threads, counting semaphores, and mutex synchronization. The output shows the producer generating ten items and the consumer consuming those items.